WIFI Network Basics

---

1. Electromagnetic Waves and Wireless Communication

1.1 What are Electromagnetic Waves

WiFi is essentially **electromagnetic waves** - changing electric fields generate magnetic fields, changing magnetic fields generate electric fields, and the two alternately propagate through space.

Key Parameter	Description
Frequency (f)	Number of oscillations per second, unit Hz. WiFi uses 2.4 GHz = 2.4 billion oscillations per second
Wavelength (λ)	Physical length of one complete wave. λ = c / f ≈ 0.125m (approximately 12.5cm at 2.4GHz)
Amplitude	Signal strength, affected by distance and obstacles
Modulation	Method of "encoding" data onto a carrier wave (amplitude/frequency/phase modulation)

> Higher electromagnetic wave frequency means shorter wavelength and weaker penetration (greater wall penetration loss), but greater data carrying capacity.

1.2 Why WiFi Uses 2.4GHz

Frequency Band	Wavelength	Characteristics
2.4 GHz	~12.5 cm	Strong wall penetration, but many devices and interference (microwave ovens, Bluetooth all use this band)
5 GHz	~6 cm	Faster speed, less interference, but weak wall penetration
6 GHz (WiFi 6E)	~5 cm	Latest band, many channels, but shortest range

ESP32-S3 **only supports 2.4GHz**. 5G WiFi routers need to enable 2.4G band simultaneously to be connected by ESP32-S3.

---

2. WiFi Operating Principles

2.1 Channel

The 2.4GHz band is divided into **14 channels** (China can use 1~13), each channel spaced 5MHz apart, but the WiFi signal itself occupies 22MHz bandwidth.

```
Channel:    1        6        11
Frequency: 2412MHz  2437MHz  2462MHz
           ████             ████        ← Non-overlapping (recommended to use 1/6/11)
```

> **Channel Selection Principle**: Adjacent routers should choose different channels from 1, 6, 11 to avoid co-channel interference.

2.2 RSSI (Signal Strength)

RSSI (Received Signal Strength Indicator) indicates how strong the signal the receiver "hears":

RSSI Range	Signal Quality	Actual Experience
> -50 dBm	Excellent	Router is right next to you
-50 ~ -65 dBm	Good	Normal use
-65 ~ -75 dBm	Fair	May have occasional stuttering
-75 ~ -85 dBm	Weak	Edge areas, prone to disconnection
< -85 dBm	Very Weak	Basically unusable

> dBm is a logarithmic unit: every 3 dB decrease halves the signal power. `-50 dBm` is 1000 times stronger than `-80 dBm`.

2.3 802.11 Protocol Evolution

Standard	Year	Frequency Band	Max Rate	Common Name
802.11b	1999	2.4 GHz	11 Mbps	—
802.11g	2003	2.4 GHz	54 Mbps	—
802.11n	2009	2.4/5 GHz	150 Mbps	WiFi 4
802.11ac	2013	5 GHz	866 Mbps	WiFi 5
802.11ax	2019	2.4/5/6 GHz	1200 Mbps	WiFi 6

ESP32-S3 supports **802.11 b/g/n**, theoretical max rate 150 Mbps (HT40 mode), actual throughput approximately 20~50 Mbps.

2.4 WiFi Security Authentication

Authentication Mode	Encryption Algorithm	Security	ESP32-S3 Support
OPEN	None	Insecure	✅
WEP	RC4	Obsolete (crackable in minutes)	✅
WPA-PSK	TKIP	Weak	✅
WPA2-PSK	**AES-CCMP**	**Secure (Recommended)**	✅
WPA3-PSK	SAE	More Secure	✅
WPA2-Enterprise	AES + RADIUS	Enterprise	✅

---

3. WiFi Operating Modes

3.1 STA Mode (Station)

ESP32 connects to an existing router like a phone, obtaining an IP address to access the internet.

```
Internet ↔ [Router/AP] ↔ WiFi ↔ [ESP32-STA]
                                  IP: DHCP dynamically assigned (e.g., 192.168.1.100)
```

Characteristics:

- IP assigned by router DHCP (not fixed, unless static IP is configured)
- Can access internet and LAN

- Foundation for all subsequent network routines (UDP/TCP/HTTP/MQTT)

3.2 AP Mode (Access Point)

ESP32 **becomes its own hotspot**, other devices connect to it.

```
                [ESP32-AP]
                IP: 192.168.4.1 (fixed)
                DHCP Server: assigns IP to connectors
               /    |    \
         [Phone]  [PC]  [Another ESP32]
        .2       .3        .4
```

Characteristics:

- IP fixed at 192.168.4.1, ESP32 acts as its own DHCP server
- Cannot access internet (unless NAT forwarding is implemented)
- Used for configuration, direct connection control, offline LAN communication

3.3 STA + AP Hybrid Mode

ESP32 operates as both Station and Access Point simultaneously:

```
   Internet ↔ [Router] ↔ [ESP32] ↔ [Phone directly connects to ESP32 hotspot]
              WiFi STA    WiFi AP
```

Typical application: **SmartConfig Provisioning** — Phone connects to ESP32 hotspot → tells it the router password → ESP32 switches to STA mode to connect to the router.

3.4 STA vs AP Comparison

Comparison	STA Mode	AP Mode
Role	Client (connect to others)	Hotspot (others connect to it)
IP Source	DHCP dynamically obtained	Fixed 192.168.4.1
DHCP Service	Provided by router	ESP32 provides its own
Max Connections	N/A	Configurable (default 4, max 10)
Network Capability	Can access internet	LAN only
Core API	`esp_wifi_connect()`	`esp_wifi_start()` (no connect needed)
Typical Scenario	IoT device internet access, data reporting	Configuration provisioning, direct control

---

4. ESP32-S3 WiFi Hardware Specifications

Feature	Parameter
WiFi Standard	IEEE 802.11 b/g/n
Operating Frequency Band	2.4 GHz
Max TX Power	+21 dBm (11b)
Max RX Sensitivity	-97 dBm (11b)
Supported Protocols	WPA/WPA2/WPA3 Personal/Enterprise
Antenna	On-board ceramic antenna / IPEX external antenna
Operating Modes	STA / AP / STA+AP / Sniffer
Channel Range	1 ~ 14

5. ESP32-S3 WiFi Development Key Points

5.1 WiFi and ADC2 Resource Conflict

ESP32-S3 has two ADC units: ADC1 and ADC2. **ADC2 shares internal hardware resources with WiFi**:

Scenario	ADC1	ADC2
WiFi Off	Normal	Normal
WiFi On	**Normal**	**Abnormal readings / Unavailable**

> In development practice, any project enabling WiFi must use ADC1 (GPIO 1~9 channel range) for analog collection.

5.2 NVS——WiFi Storage Dependency

WiFi driver needs to read RF calibration parameters stored in NVS (Non-Volatile Storage) during initialization. **Regardless of whether WiFi configuration is saved, `nvs_flash_init()` is required**.

WiFi driver depends on RF calibration parameters stored in NVS; NVS partition must be initialized before startup — if the partition is full or version incompatible, erase and reinitialize.

```
// Initialize NVS flash; WiFi driver depends on RF calibration parameters stored in it
esp_err_t ret = nvs_flash_init();
if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
    // NVS partition full or version mismatch — erase and retry
    ESP_ERROR_CHECK(nvs_flash_erase());
    ret = nvs_flash_init();
}
```

5.3 WiFi Initialization Fixed Sequence

ESP-IDF WiFi initialization sequence is **strictly non-adjustable**; skipping any step causes crash:

WiFi initialization has strict sequence requirements; the following ten steps are all essential, and reversing the order will cause startup failure.

```
① nvs_flash_init()           ← NVS storage
② esp_netif_init()           ← TCP/IP network stack initialization
③ esp_event_loop_create_default() ← Event loop
④ esp_netif_create_default_wifi_sta()  (or _ap())  ← Network interface
⑤ esp_wifi_init(&cfg)        ← WiFi driver
⑥ Register event callback              ← WIFI_EVENT + IP_EVENT
⑦ esp_wifi_set_mode()        ← STA / AP / APSTA
⑧ esp_wifi_set_config()      ← SSID + Password
⑨ esp_wifi_start()           ← Start WiFi
⑩ esp_wifi_connect()         ← STA mode initiates connection (not needed for AP mode)
```

> If `esp_wifi_connect()` is called immediately after `esp_wifi_start()`, it may cause crash; it is recommended to add `vTaskDelay(500ms)` to wait for lower layer to be ready.

5.4 Antenna

ESP32-S3 development boards typically use **on-board ceramic antenna** (the small gray square on the PCB). Some core boards provide **IPEX connector** for external high-gain antenna:

Antenna Type	Gain	Applicable Scenario
On-board ceramic antenna	~2 dBi	Short range, general use
IPEX external rod antenna	~5 dBi	Wall penetration, long distance

> When using IPEX external antenna, need to configure antenna selection GPIO in menuconfig or call `esp_wifi_set_ant_gpio()` in code.

5.5 Strapping Pins

ESP32-S3's **GPIO0, GPIO3, GPIO45, GPIO46** are Strapping pins (determine chip operating mode at startup), with internal pull-up/pull-down resistors. **These pins cannot be used for ADC input**, because internal resistors will raise/lower the voltage level making readings always close to VCC or GND.

5.6 WiFi Event-Driven Model

ESP-IDF WiFi uses **event-driven architecture**. WiFi state changes generate events from the lower layer, and developers register callback functions to handle them:

When WiFi lower-layer state changes, corresponding events are triggered, and callback functions distribute to different handling branches based on event type.

```
WiFi state change → WIFI_EVENT / IP_EVENT → event_handler()
                                    ├─ WIFI_EVENT_STA_START       → Auto connect
                                    ├─ WIFI_EVENT_STA_DISCONNECTED → Clear flags + reconnect
                                    ├─ WIFI_EVENT_SCAN_DONE        → Print AP list
                                    ├─ WIFI_EVENT_AP_START         → Hotspot created
                                    ├─ WIFI_EVENT_AP_STACONNECTED  → Device connected
                                    ├─ WIFI_EVENT_AP_STADISCONNECTED → Device disconnected
                                    └─ IP_EVENT_STA_GOT_IP       → Set flag + print IP
```

5.7 EventGroup Synchronization Mechanism

WiFi connection is **asynchronous** — `esp_wifi_connect()` returns immediately, actual connection result is notified by event callback. Use **FreeRTOS Event Group** to implement synchronous waiting:

WiFi connection is asynchronous; main thread blocks and waits for connection result via event group, callback sets bits to notify and unblock.

```
// Main thread: block and wait for WiFi connection success
xEventGroupWaitBits(wifi_event_group,   // Event group handle
                    WIFI_CONNECTED_BIT, // Event bit to wait for
                    // Don't clear the bit on exit
                    pdFALSE,
                    // Wait for any bit (not all)
                    pdFALSE,
                    timeout);           // Block timeout / timeout in ticks

// Got IP, set bit, wake main
xEventGroupSetBits(wifi_event_group, WIFI_CONNECTED_BIT);
```

---

6. WiFi OTA Firmware Upgrade

6.1 What is OTA

OTA (Over-The-Air) is one of the most important WiFi applications — devices download new firmware via WiFi, write it to Flash and restart to switch, all without USB cable or JTAG programmer.

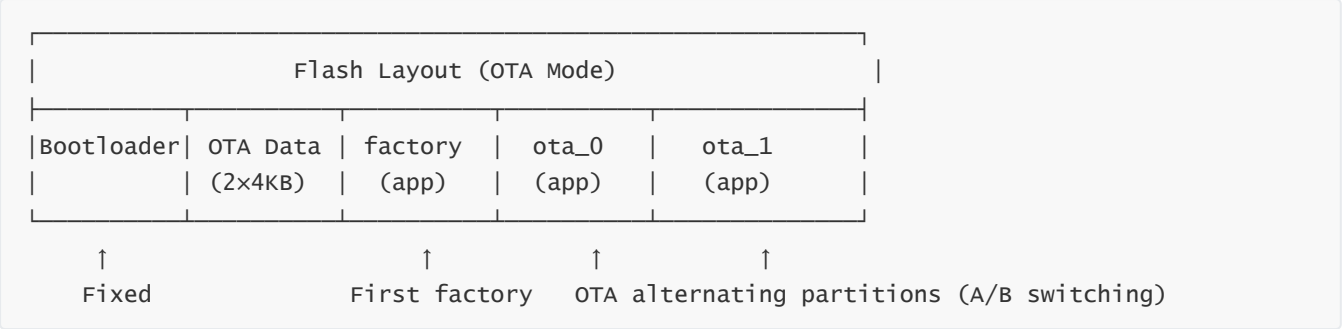
Upgrade Method	Operation	Applicable Stage	Requires Physical Wiring
USB Serial Flash	USB cable + idf.py flash	Development/Debugging	Yes
JTAG Flash	JTAG debugger	Development/Debugging	Yes
WiFi OTA	**HTTP download + partition switch**	**Mass production**	**No**

> OTA is a **core operational capability** for IoT products — deployed devices can be repaired and have features added without retrieval.

6.2 Flash Partition Layout

OTA requires at least two APP partitions used alternately (A/B switching) to prevent bricking from power failure during upgrade:

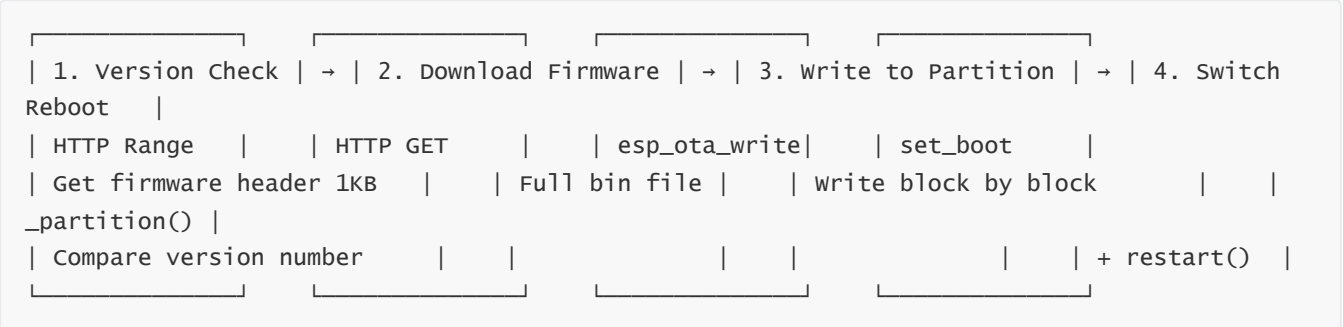
Below is the Flash partition layout in OTA mode; factory stores factory firmware, ota_0 and ota_1 alternately write new firmware, OTA Data records metadata of the partition for next boot.



- **factory**: Factory firmware, written during mass production flashing
- **ota_0 / ota_1**: Two partitions used alternately. Currently running ota_0 → OTA writes to ota_1 → Reboot and boot from ota_1 → Next OTA writes to ota_0
- **OTA Data**: Records metadata of "which partition to boot from next" (2 sectors backup each other)

> **Why are two APP partitions needed?** If power is cut off while writing new firmware, the old partition is unaffected, and the device can still run the old version after restart. This is the OTA "anti-bricking" mechanism.

6.3 OTA Upgrade Process



Step 1 — Version Check: Through HTTP Range request, only download the first 1024 bytes of firmware, search for `esp_app_desc_t` magic number (`0xABCD5432`) to locate version number string, compare with local `esp_app_get_description()->version`. If server version ≤ local version, skip upgrade.

Step 2 — Download Firmware: After confirming new version, HTTP GET full `.bin` firmware file.

Step 3 — Write Block by Block: `esp_ota_begin()` opens target partition → loop `esp_http_client_read()` + `esp_ota_write()` write → `esp_ota_end()` close and verify.

Step 4 — Switch Boot: `esp_ota_set_boot_partition()` modifies OTA Data to point to new partition → `esp_restart()` reboot.

6.4 Version Number Extraction — Magic Number Search Method

`esp_app_desc_t` structure layout in bin file:

Firmware header contains magic number identifier and version number field; version information is located by searching fixed byte sequence — magic number 0xABCD5432 at offset 0x00, 32-byte version number string at offset 0x10.

```
Offset     Content
──────────────────────────────────────────
0x00       magic_word = 0xABCD5432          ← Magic number (little-endian: 32 54 CD AB)
0x04       secure_version
0x08       Reserved field
0x10       version[32]                      ← Version number string, e.g., "1.0.0"
...
```

Search process:

1. HTTP Range `bytes=0-1023` download firmware header 1KB
2. Byte-by-byte matching `{0x32, 0x54, 0xCD, 0xAB}`
3. Magic number offset +0x10 → read 32-byte version number
4. Verify first character is '0'~'9'

6.5 OTA Core API

OTA operation follows the six-step process: "get partition → begin write → write block by block → end verify → set boot partition → reboot switch", each step corresponds to a core API.

```c
// Get the "next" writable OTA partition (the one outside the currently running partition)
const esp_partition_t *esp_ota_get_next_update_partition(NULL);

// Begin OTA write (OTA_SIZE_UNKNOWN = unknown firmware size)
esp_ota_begin(partition, OTA_SIZE_UNKNOWN, &handle);

// Write in chunks (4KB max)
esp_ota_write(handle, data, size);

// End write (verify+close)
esp_ota_end(handle);

// Set next boot partition (modify OTA Data metadata)
esp_ota_set_boot_partition(partition);

// Reboot to new firmware
esp_restart();
```

6.6 Firmware Server

OTA requires an HTTP file server to host `.bin` files. During testing, you can start a simple server on your PC:

PC side executes one command to start HTTP server; ESP32 accesses this address via WiFi to download firmware.

```
# Start HTTP server with Python 3 (port 8080, root = current directory)
python -m http.server 8080

# On Windows, ensure the firewall allows port 8080
```

ESP32 side `OTA_URL` points to `http://<PC_IP>:8080/WiFi_OTA.bin`.

6.7 OTA Precautions

Key Point	Description
Partition table must include OTA partitions	menuconfig → Partition Table → "Factory app, two OTA definitions"
Firmware size limit	ota_0/ota_1 each occupy half of Flash minus factory; if exceeded, `esp_ota_write` fails
Network stability	If WiFi disconnects during download → `esp_ota_abort()` rollback, device still runs old version
Version number format	Numbers and dots only (`1.0.0`), do not include `v` prefix
Security	Production environment should use HTTPS + certificate verification + firmware signing to prevent man-in-the-middle attacks
Rollback	ESP-IDF supports automatic rollback — if new firmware restarts continuously N times, it automatically switches back to old version

> For complete code implementation and operation steps, see (5.Network Protocols and Wireless Communication->12.WIFI-OTA Upgrade->WIFI-OTA Upgrade).

7. Common WiFi Troubleshooting

Symptom	Cause	Solution
Continuously prints "Disconnected, reconnecting..."	SSID or password incorrect	Check `WIFI_SSID` and `WIFI_PASS` macros
Connection to 5G WiFi fails	ESP32-S3 only supports 2.4G	Ensure router 2.4G band is enabled
Got IP but ping cannot reach external network	Router not connected to internet / DNS issue	First ping router IP to confirm LAN is working

Symptom	Cause	Solution
`esp_wifi_connect()` returns error	WiFi not initialized completely	Ensure `vTaskDelay(500ms)` is added after `esp_wifi_start()`
`WIFI_EVENT_STA_DISCONNECTED` triggers frequently	Weak signal or congested router channel	Move closer to router / change WiFi channel
Compilation error `nvs_flash.h` not found	NVS not initialized	Add `nvs_flash_init()` at the beginning of `app_main()`
AP mode phone cannot find hotspot	Channel occupied / not started properly	Check logs for "WiFi AP started"; try changing `AP_CHANNEL`
AP mode connection shows "password incorrect"	Password less than 8 characters	WPA2 requires password ≥ 8 characters

Common Network Communication Protocols

> **Importance**: TCP/IP protocol stack is the fundamental cornerstone of all network communication for ESP32-S3 — regardless of whether you use WiFi, Ethernet, HTTP, MQTT, or OTA upgrade, data ultimately transmits over the network through the TCP/IP protocol stack. Understanding the layered model, IP addressing, port mechanism, Socket API, and core differences between TCP/UDP is a prerequisite for troubleshooting network issues, optimizing communication performance, and designing reliable IoT applications. This chapter does not involve specific code flashing, but establishes a **theoretical framework** for network communication — all API calls, parameter configurations, and exception troubleshooting in subsequent network routines (UDP/TCP/HTTP/MQTT) are based on the knowledge in this chapter.

1. TCP/IP Protocol Stack

1.1 Layered Model

The essence of network communication is **layered encapsulation, peer-to-peer communication**:

```
┌──────────────────────────────────────────────┐
|              Application Layer               |  HTTP / MQTT / DNS / OTA
|  http://example.com/api  →  "Hello ESP32"    |
├──────────────────────────────────────────────┤
|              Transport Layer                 |  TCP / UDP
|  Port number (80/8080/1883) + reliable/unreliable transmission        |
├──────────────────────────────────────────────┤
|              Internet Layer                  |  IP
|  IP address (192.168.1.100) + routing + fragmentation        |
├──────────────────────────────────────────────┤
|              Link Layer                      |  WiFi / Ethernet
|  MAC address (AA:BB:CC:DD:EE:FF) + channel access      |
└──────────────────────────────────────────────┘
```

1.2 Data Encapsulation Process

When sending data, each layer adds its own header, like **Russian nesting dolls**:

```
Application layer:  [ HTTP GET /index.html ]
                    ↓ Add TCP header
Transport layer:  [ TCP header | src_port=12345 dst_port=80 | HTTP GET ... ]
                    ↓ Add IP header
Network layer:  [ IP header | src=192.168.1.100 dst=93.184.216.34 | TCP header | HTTP GET
... ]
                    ↓ Add WiFi frame header
Link layer:  [ WiFi frame header | src=AA:BB:CC dst=DD:EE:FF | IP header | TCP header | HTTP
GET ... | FCS checksum ]
```

The receiver de-encapsulates layer by layer until the application layer gets the original data.

1.3 Key Protocol Comparison

Protocol	Layer	Reliability	Connection	Header Overhead	Typical Use
IP	Network Layer	Unreliable	Connectionless	20B	Addressing, routing
ICMP	Network Layer	Unreliable	Connectionless	8B	Ping, error reporting
TCP	Transport Layer	**Reliable**	**Connection-oriented**	20B	Web, files, MQTT
UDP	Transport Layer	Unreliable	Connectionless	8B	DNS, video streaming, sensors
ARP	Link Layer	—	—	28B	IP→MAC address resolution
DHCP	Application Layer	—	—	—	Automatic IP acquisition

2. IP Address

IP address is the "house number" for network communication — after ESP32 obtains an IP, it can be accessed by other devices on the LAN, which is a prerequisite for all subsequent Socket communication, HTTP requests, and MQTT connections.

2.1 IPv4 Basics

IPv4 address consists of **32 bits**, written as 4 decimal octets (each 0~255):

```
192     .   168     .   1       .   100
11000000  10101000  00000001  01100100     ← 32-bit binary
└──────────────────┬──────────────────┘└──┬──┘
          Network part          Host part
       (subnet mask determines boundary)
```

2.2 Subnet Mask

Subnet mask determines which part of an IP address is the "network number" and which part is the "host number":

Subnet Mask	CIDR	Available IPs	Common Scenario
255.255.255.0	/24	254	Home router
255.255.0.0	/16	65534	Large enterprise

```
IP:      192.168.  1.100
Mask:   255.255.255.  0
        ──────────  ───
         Network part   Host part (0~255, where 0=network number, 255=broadcast)
```

> All devices in the same network have the same "network part" and different "host parts".

2.3 Public IP vs Private IP

Range	Type	Description
10.0.0.0 ~ 10.255.255.255	Private Class A	Large enterprise internal
172.16.0.0 ~ 172.31.255.255	Private Class B	Medium enterprise internal
192.168.0.0 ~ 192.168.255.255	**Private Class C (Home Ethernet)**	**Most common — home router**
Rest	Public IP	Unique on the internet, requires purchase

> Private IP cannot be accessed directly from the internet — router maps private IP to public IP through **NAT (Network Address Translation)**.

2.4 Special IP Addresses

IP	Meaning	Use
`127.0.0.1`	Localhost	Access yourself
`0.0.0.0`	Any address	Bind to all network interfaces
`255.255.255.255`	Broadcast address	Send to all devices on LAN
`192.168.1.1`	Usually router gateway	Management page entry

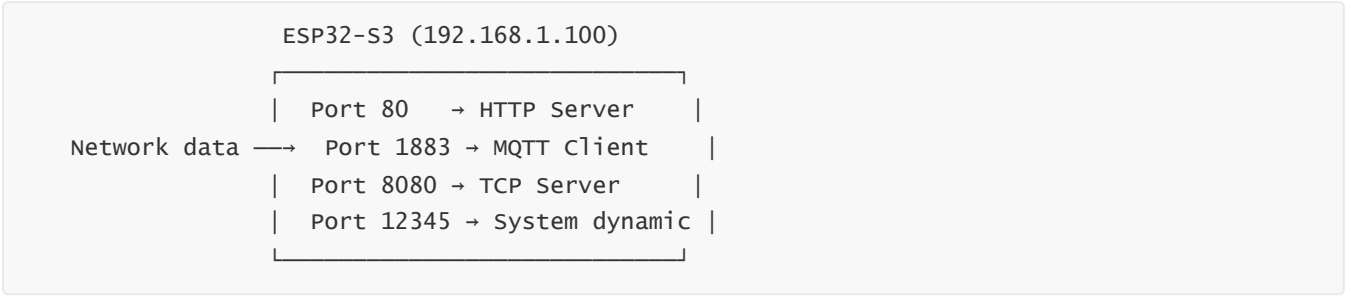
---

3. Port

Port is the key to achieving "one device, multiple services" — ESP32 can simultaneously run HTTP server (80), MQTT client (1883), and TCP data channel (8080), each service binding to a different port without interference. Firewall and NAT configuration also depend on port numbers.

3.1 Concept of Port

IP address locates a device, port locates a program on the device.



Port is a 16-bit number (0~65535):

Range	Type	Description
0 ~ 1023	Well-known ports	HTTP=80, HTTPS=443, MQTT=1883, DNS=53
1024 ~ 49151	Registered ports	Custom services (e.g., 8080)
49152 ~ 65535	Dynamic ports	Client temporary use

3.2 Client Port vs Server Port

```
Client (browser/ESP32)                    Server
————————————————————————        ————————————————————
socket()                                  socket()
(System automatically assigns temporary port,                  bind(80)          ← Must be
fixed port,
 like 52341)                                                 client knows which port to
connect to
connect(93.184.216.34:80) ————————→  listen(80) + accept()
                                    Connection received: client IP:52341 ↔ server IP:80
send() ——————————————————————————→  recv() → handle → send()
recv() ←——————————————————————————  (reply)
```

> **Server must `bind()` a fixed port**, otherwise client doesn't know where to connect; client port is automatically assigned by the operating system (used and discarded).

---

4. Socket Programming Basics

Socket is the **core API layer** for ESP-IDF network programming — whether it's custom TCP/UDP communication, HTTP client, or MQTT client, the lower layer all sends and receives data through Socket. Mastering Socket data structures, byte order conversion, and creation methods is the foundation for reading and writing all network code.

4.1 What is Socket

Socket is a network programming interface provided by the operating system — developers send and receive data through Socket without caring about lower-layer details like WiFi frames, IP fragmentation, TCP retransmission, etc.

```
Application code                    Operating system kernel
————————————                    ————————————————

send(sock, "Hello", 5)
    |                           ┌————————————————┐
    └———————————————————————→   TCP protocol stack | → segmentation → add TCP header
                                ├————————————————┤
                                |   IP protocol stack  | → fragmentation → add IP header →
routing
                                ├————————————————┤
                                |  WiFi driver   | → add WiFi frame → DMA → antenna transmit
                                └————————————————┘
```

4.2 Key Data Structures

The two most important address structures in Socket programming — `sockaddr` is a generic type, `sockaddr_in` is an IPv4-specific type, all Socket APIs use the latter through casting.

```
// Generic socket address (IPv4/IPv6)
struct sockaddr {
```

```
    // Address family: AF_INET(IPv4) or AF_INET6(IPv6)
    sa_family_t sa_family;
    char        sa_data[14];  // Protocol address (IP + Port)
};
struct sockaddr_in {
    sa_family_t    sin_family; // AF_INET / Address family: IPv4
    // Port number (network byte order)
    uint16_t       sin_port;
    struct in_addr sin_addr;   // IP address
};
struct sockaddr_in server_addr;
bind(sock_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
```

4.3 Network Byte Order

Network byte order = Big-Endian, meaning the high-order byte is stored in the low address. ESP32 CPU is Little-Endian, opposite of network order, must convert:

```
Number 0x12345678 (32-bit)

Big-Endian (Network order):
Low address ← → High address
 12 | 34 | 56 | 78     ← High-order byte first

Little-Endian (ESP32 host order):
Low address ← → High address
 78 | 56 | 34 | 12     ← Low-order byte first
```

Function	Meaning	Use
`htons(n)`	Host TO Network Short (16-bit)	Port number conversion
`htonl(n)`	Host TO Network Long (32-bit)	IP address conversion (less common)
`ntohs(n)`	Network TO Host Short	Restore after receiving
`ntohl(n)`	Network TO Host Long	Restore after receiving
`inet_pton()`	String → binary IP (automatic network order)	`"192.168.1.1"` → `0x0101A8C0`

Both port and IP address need to be converted to network byte order — `htons()` converts port, `inet_pton()` converts IP string to network-order binary.

```
// Correct: use conversion functions
// Convert to network byte order
addr.sin_port = htons(8080);
// String → binary (network order)
inet_pton(AF_INET, "192.168.1.100", &addr.sin_addr);

// Wrong: direct assignment
// ❌ Little-endian, network sees 0x901F
addr.sin_port = 8080;
```

4.4 Socket Creation Types

`socket()` differentiates TCP (stream) from UDP (datagram) through the `type` parameter — this is the first step for all subsequent Socket operations, determining the communication mode.

```
int fd = socket(domain, type, protocol);  // General form
int tcp_sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
int udp_sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
```

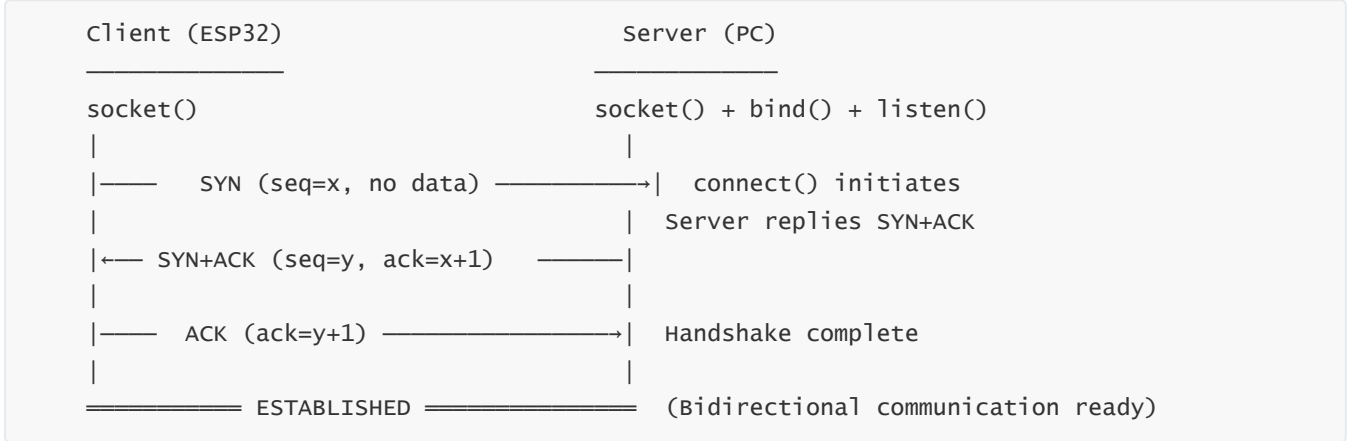
Parameter	TCP	UDP
domain	AF_INET (IPv4)	AF_INET (IPv4)
type	**SOCK_STREAM** (stream)	**SOCK_DGRAM** (datagram)
Behavior	Connection-oriented, reliable, byte stream	Connectionless, unreliable, preserves message boundaries

---

5. TCP In-Depth

TCP is the most commonly used reliable transmission protocol for IoT devices — firmware OTA upgrade cannot lose a single byte, MQTT messages must be reliably delivered, HTTP webpages must be completely loaded; all these scenarios depend on TCP's acknowledgment and timeout retransmission mechanisms. Understanding three-way handshake, four-way wave, and `recv()` return values is the foundation for stable TCP server operation.

5.1 Three-Way Handshake



- SYN: Request to establish connection
- SYN+ACK: Agree + request (server also confirms its direction)
- ACK: Acknowledge (officially established)

> The essence of three-way handshake is **both parties confirming each other's send/receive capability is normal**.

5.2 Four-Way Wave

```
    Client                                  Server
    ————————                                ————————
    close() or shutdown()
    |———— FIN (seq=u) ————————————————————→|  Client: "I'm done sending data"
    |←———— ACK (ack=u+1)  ————————————————|  Server: "Got it"
    |                                       |  (Server may still have data to send)
    |←———— FIN (seq=v)  ————————————————————|  Server: "I'm done too"
    |———— ACK (ack=v+1) ————————————————————→|  Client: "Got it"
    |                                       |
    ═══════ Connection Closed ═══════════════
```

> TCP is full-duplex — each direction closes independently. `close()` closes both directions; `shutdown(fd, SHUT_WR)` only closes the write direction (can still continue receiving).

5.3 How Reliable Transmission is Achieved

TCP's reliability comes from three mechanisms:

Mechanism	Principle
Acknowledgment (ACK)	Reply ACK after receiving data; sender retransmits if ACK not received
Sequence Number (SEQ)	Each byte has a number; receiver sorts and reassembles by sequence number
Timeout Retransmission (RTO)	Start timer after sending; retransmit if ACK not received within timeout

5.4 Three Cases of `recv()` Return Value

Return Value	Meaning	Handling
`> 0`	Received n bytes of data	Normal processing
`= 0`	**Peer closed connection normally** (sent FIN)	`close()` and `break`
`< 0`	No data available (`MSG_DONTWAIT`) or error	Check `errno`

`recv()` three return values correspond to three handling branches — this is the standard template for TCP receiving logic.

```c
int ret = recv(sock_fd, buf, len, 0);
if (ret > 0)     { /* Data received, process normally */ }
else if (ret == 0) { /* Peer closed, break + close() */ }
else             { /* No data (non-blocking), or real error */ }
```

5.5 TCP Server Standard Process

```
socket() → bind() → listen() → while(1) {
    client_fd = accept() → while(1) {
        recv() / send()    ← Communicate with single client
    } → close(client_fd)   ← Client disconnected, back to accept
}
```

---

6. UDP In-Depth

UDP's low overhead and low latency characteristics make it the preferred choice for high-frequency sensor data reporting, real-time audio/video streaming, DNS queries, and other scenarios. In IoT, when "speed takes priority over reliability" — such as temperature/humidity collection hundreds of times per second, device discovery broadcasts — UDP is more suitable than TCP.

6.1 Meaning of Connectionless

UDP doesn't handshake, doesn't establish connection, doesn't guarantee delivery — datagrams are like **throwing a drift bottle**:

```
     Client (ESP32)                              Server (PC)
     ──────────────                              ────────────
     socket(SOCK_DGRAM)                          socket(SOCK_DGRAM) + bind(8080)
     |                                            |
     |── sendto("Hello", 192.168.1.183:8080) ──→|  Send directly
     |                                            |  (received if received, not received if
not)
     |←─ sendto("Reply", 192.168.1.100:52341) ──|  Reply
```

6.2 UDP Applicable Scenarios

✅ Suitable for UDP	❌ Not Suitable for UDP
Real-time video/audio streaming (occasional glitching acceptable)	File transfer (cannot lose a single byte)
DNS queries (small request volume, timeout retry works)	Web browsing (HTML must be complete)
High-frequency sensor data (losing a few frames is fine)	Firmware OTA upgrade (firmware must be complete)

✅ Suitable for UDP	❌ Not Suitable for UDP
Broadcast/multicast messages	Financial transactions (packet loss = money loss)

6.3 UDP vs TCP API Comparison

TCP and UDP API call flow comparison — TCP requires `listen()` + `accept()` to establish connection, then communicates via file descriptor; UDP directly uses `sendto()` / `recvfrom()` with target address.

```
// ===== TCP Server =====          // ===== UDP Server =====
socket(AF_INET, SOCK_STREAM, 0)             socket(AF_INET, SOCK_DGRAM, 0)
bind(fd, addr, len)                         bind(fd, addr, len)
listen(fd, 5)                               // No listen / No listen
// No accept, direct recvfrom / No accept
client_fd = accept(fd, ...)
recv(client_fd, buf, len, 0)                recvfrom(fd, buf, len, 0, &addr, &len)
send(client_fd, buf, len, 0)                sendto(fd, buf, len, 0, &addr, len)

// ===== TCP Client =====          // ===== UDP Client =====
socket(AF_INET, SOCK_STREAM, 0)             socket(AF_INET, SOCK_DGRAM, 0)
connect(fd, &addr, len)                     // No connect / No connect
send(fd, buf, len, 0)                       sendto(fd, buf, len, 0, &addr, len)
recv(fd, buf, len, 0)                       recvfrom(fd, buf, len, 0, &addr, &len)
```

6.4 Non-Blocking Receive (MSG_DONTWAIT)

`MSG_DONTWAIT` flag makes `recvfrom()` / `recv()` **non-blocking** — returns -1 immediately when no data (`errno=EAGAIN`) without blocking the task, suitable for high-frequency polling scenarios (e.g., low-latency receive loop every 10ms).

```
int recv_len = recvfrom(sock_fd, recv_buf, sizeof(recv_buf)-1,
                        MSG_DONTWAIT,       // Non-blocking flag
                        (struct sockaddr *)&from_addr, &len);
if (recv_len > 0) {
    // Data received, process it
}
```

---

7. HTTP Protocol Basics

HTTP is the standard protocol for IoT devices to interact with cloud REST APIs — devices use GET to fetch configuration, POST to report sensor data, Range requests to implement OTA firmware chunked download. Understanding request/response format and status codes is a prerequisite for calling Web APIs.

7.1 Request Format

```
GET /index.html HTTP/1.1\r\n          ← Request line: method + URI + version
Host: example.com\r\n                 ← Request header: key-value pairs
User-Agent: ESP32\r\n

Accept: */*\r\n

\r\n                                  ← Empty line = end of headers
                                       ← Request body (empty for GET, has content for POST)
```

7.2 Response Format

```
HTTP/1.1 200 OK\r\n                    ← Status line: version + status code + reason phrase
Content-Type: text/html\r\n            ← Response header
Content-Length: 125\r\n

Connection: close\r\n

\r\n                                   ← Empty line = end of headers
<html>...response body...</html>               ← Response body
```

7.3 Common Status Codes

Status Code	Meaning	Description
200	OK	Request successful
206	Partial Content	Range request returns partial content (OTA firmware header download)
301	Moved Permanently	Permanent redirect
302	Found	Temporary redirect
400	Bad Request	Request format error
404	Not Found	Resource does not exist
500	Internal Server Error	Server internal error

7.4 Common HTTP Methods

Method	Semantics	Request Body	Typical Use
GET	Get resource	None	Query data, fetch webpage

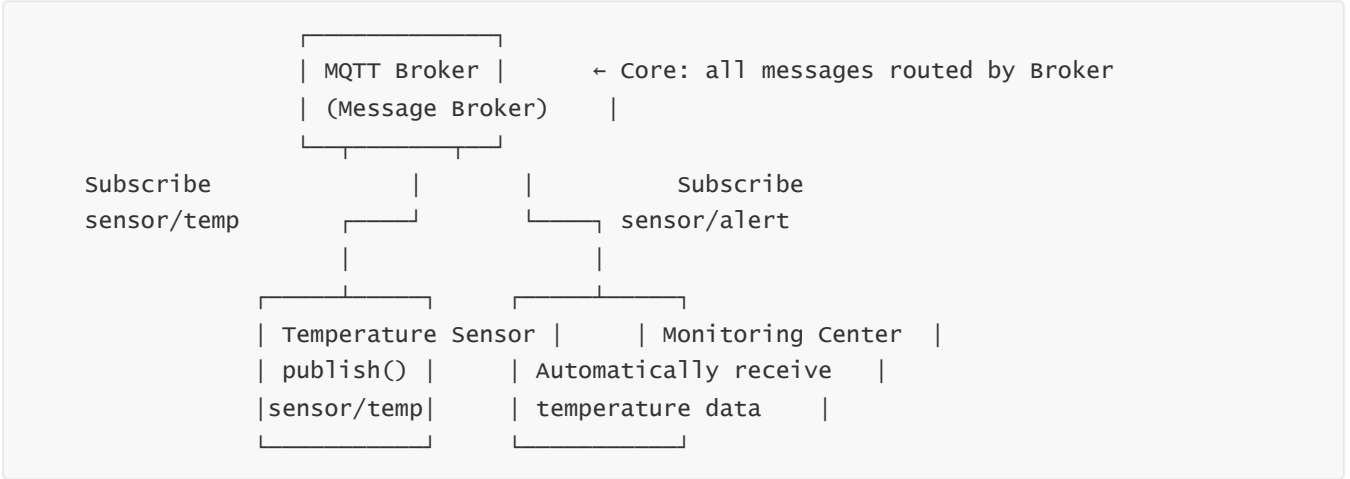
Method	Semantics	Request Body	Typical Use
POST	Submit data	Yes	Submit form, upload sensor data
PUT	Replace resource	Yes	Update configuration
DELETE	Delete resource	None	Delete record

8. MQTT Protocol Basics

8.1 Why IoT Uses MQTT Instead of HTTP

Comparison	MQTT	HTTP
Communication Mode	Publish/Subscribe (Broker relay)	Request/Response (direct connection)
Message Push	Native support, subscribe and push	Requires client polling (or WebSocket)
Minimum Protocol Header	2 bytes	Hundreds of bytes
Connection Keep-alive	Keep Alive heartbeat	Stateless (each request independently connects)
One-to-Many	Naturally supported (multiple people subscribe to same Topic)	Requires server polling + push logic

8.2 Publish/Subscribe Model



Three Roles:

Role	Description	Example
Publisher	Send message to specific Topic	ESP32 reports temperature

Role	Description	Example
Subscriber	Subscribe to Topic, automatically receive messages	Phone App subscribes to sensor data
Broker	Relay all messages, manage Topic subscription relationships	EMQX, Mosquitto

8.3 Topic Wildcards

Wildcard	Semantics	Example
`+`	Match **single level**	`home/+/light` → `home/bedroom/light`
`#`	Match **multiple levels**	`home/#` → `home/bedroom/light/status`

8.4 QoS Three Levels

QoS	Semantics	Mechanism
0	At most once (fire and forget)	No retry, no acknowledgment
1	At least once (may duplicate)	Send → wait for PUBACK → timeout retransmit
2	Exactly once (no duplicate)	PUBREC → PUBREL → PUBCOMP four-way handshake

---

9. ESP-IDF Network API Selection Guide

When developing network applications, ESP-IDF provides APIs **from bottom to top layers**; select according to needs:

```
High-level (simple, good functional encapsulation)
  |  esp_http_client / esp_http_server    ← HTTP request/server
  |  mqtt_client                          ← MQTT client
  |  esp_ota                              ← OTA upgrade
  |
  |  lwip/sockets (BSD Socket API)        ← TCP/UDP raw communication
  |    socket / bind / listen / accept / connect
  |    send / recv / sendto / recvfrom
  |
Low-level (flexible, need to manually handle details)
  |  esp_wifi                             ← WiFi connection/scan/configuration
  |  esp_netif                            ← Network interface management
  |  esp_event                            ← Event-driven framework
```

If you want to...	Use this API
Connect to WiFi	`esp_wifi` + `esp_netif`

If you want to...	Use this API
Custom TCP/UDP communication	`lwip/sockets` (BSD Socket)
Access webpage / REST API	`esp_http_client`
Build Web server	`esp_http_server`
IoT sensor reporting / remote control	`mqtt_client`
WiFi remote firmware upgrade	`esp_ota` + `esp_http_client`
Ping remote host	`lwip/ping.h` or `esp_ping`

---

10. Common Debugging Methods

Network troubleshooting uses a **layer-by-layer localization strategy** — confirm from bottom to top: WiFi connection → IP acquisition → LAN connectivity → port listening → application layer protocol. Skipping layers easily leads to confusion.

10.1 Troubleshooting Steps

```
Problem: ESP32 cannot communicate

1. First check if serial logs show "Got IP: xxx.xxx.xxx.xxx"
   ├─ No → WiFi not connected → check SSID/password/frequency band
   └─ Yes → WiFi normal, continue troubleshooting

2. Ping test
   ├─ ping ESP32 IP     → Verify IP layer is working
   ├─ ping router IP    → Verify LAN connectivity
   └─ ping 8.8.8.8      → Verify external network connectivity (DNS unrelated, pure IP)

3. Port test
   ├─ On PC side use netstat -an | findstr 8080 to check if port is listening
   └─ Use network debugging assistant to send data to ESP32 IP:PORT

4. Firewall check
   └─ Windows firewall may block port 8080 and other non-standard ports
```

10.2 Common Debugging Commands

Common network debugging commands on PC side and ESP32 side — first confirm network connectivity from PC side, then confirm running status from ESP32 logs.

```
# PC side (Windows PowerShell)
ipconfig                    # Check local IP
ping 192.168.1.100          # Ping ESP32
netstat -an | findstr 8080  # Check port 8080 status
curl http://192.168.1.100/  # Test HTTP server

# ESP32 side (add in code)
ESP_LOGI(TAG, "Got IP: " IPSTR, IP2STR(&ip));  // Print obtained IP
// Check free memory
ESP_LOGI(TAG, "Free heap: %lu", esp_get_free_heap_size());
```

11. Common Network Problems

Symptom	Cause	Solution
`connect()` returns -1	Server not started or IP incorrect	Confirm PC-side Server is listening, IP and port are correct
`bind()` fails	Port already in use	Use a different port number / wait 2 minutes (TIME_WAIT timeout)
`recv()` returns 0	Peer actively closed connection	Normal disconnect, `break` then `close()`
`recv()` data "sticks" together	TCP is byte stream, no message boundaries	Application layer custom frame header + length or delimiter protocol
`send()` continuous failures	Peer closed / network interrupted	TCP connection disconnected, need to `reconnect()`
Chinese characters garbled	Terminal encoding inconsistency	Check serial assistant and network debugging assistant encoding settings (UTF-8)
PC cannot receive ESP32 data	ESP32 IP changed / firewall blocked	Check ESP32 actual IP via serial; PC firewall allows corresponding port
HTTP client `HTTP request error`	URL unreachable or DNS resolution failed	Ensure ESP32 can access external network; or use IP instead of domain name
MQTT `MQTT_EVENT_CONNECTED` not triggered	Broker address incorrect or WiFi no external network	`ping broker.emqx.io` to test external network connectivity